

Python — Gap Topics

Self-Test Exercise Book

Companion to the Gap Topics Self-Learning Guide

Name:		Date:	
Total / 324:		Time taken:	

#	Topic	Parts	Marks	Score
1	Lambda, filter(), map(), reduce()	A, B, C, D, E	34	
2	Advanced Function Arguments & Docstrings	A, B, C, D, E, F	39	
3	String Methods (title, swapcase, maketrans...)	A, B, C, D, E	42	
4	Tuples — Packing, Unpacking & Operations	A, B, C, D, E	29	
5	Sets — frozenset, Comprehension, all(), any()	A, B, C, D, E	29	
6	Dictionary Comprehension & Advanced Dicts	A, B, C, D, E	34	
7	Modules — dir(), Namespaces, LEGB	A, B, C, D, E	29	
8	Scope, Lifetime, global, nonlocal, Closures	A, B, C, D, E, F	35	
	TOTAL		324	

Instructions: 8 topics, each with MCQ, Output Prediction, Fill-in-the-Blanks, Error Fix, and Write Programs. Yellow cells = your answer. Complete each topic fully before checking the Answer Key at the back. Answer key starts after all 8 test topics.

Topic 1 — Lambda, filter(), map(), reduce()

Self-Test · 34 marks

Section	Marks	Score	Notes
A — MCQ	8		
B — Output Predict	8		
C — Fill Blanks	4		
D — Error Fix	4		
E — Write Programs	10		
TOTAL	34		

Part A — Multiple Choice [2 marks each]

Q1. What does `list(filter(None, [0, 1, "", "hi", False]))` return?

(A) [0, 1, "", "hi", False] (B) [1, 'hi'] (C) [None] (D) Error

Your answer: ()

Q2. What is the output of `list(map(lambda x,y: x+y, [1,2], [10,20]))`?

(A) [11, 22] (B) [1, 2, 10, 20] (C) [[1,10],[2,20]] (D) Error

Your answer: ()

Q3. Which import is required to use `reduce()`?

(A) from itertools import reduce (B) from functools import reduce (C) import reduce (D) No import needed

Your answer: ()

Q4. `reduce(lambda a,b: a-b, [10,2,1])` evaluates as:

(A) (10-2)-1 = 7 (B) 10-(2-1) = 9 (C) 10-2-1 = 7 (D) -(10+2+1) = -13

Your answer: ()

Part B — Fill in the Blanks [1 mark each]

- `filter()` returns a _____ — you must wrap it with `list()` to see the values.
- `map()` applies a function to _____ element of the iterable.
- `reduce()` reduces an entire list to _____ value.
- A lambda can have multiple arguments but only _____ expression.

Part C — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix the code to correctly compute the sum of squares of even numbers.

```
from functools import reduce
nums = [1,2,3,4,5,6]
result = reduce(lambda a,b: a+b,
map(lambda x: x**2,
filter(lambda x: x%2, nums)))
print(result)
```

Bug(s):

Corrected code:

Part D — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Using only `lambda`, `map()`, `filter()`, and `reduce()`, write a one-expression pipeline that: takes a list of numbers, keeps only those divisible by 3, squares them, then returns their product.

Hint: Chain: `reduce(lambda..., map(lambda..., filter(lambda..., nums)))`

Q2. Write a function `apply_all(funcs, value)` that takes a list of functions and a value, and applies them in sequence (output of one is input of the next) using `reduce()`.

Hint: `reduce(lambda result, fn: fn(result), funcs, value)`

Topic 2 — Advanced Function Arguments & Docstrings

Self-Test · 39 marks

Section	Marks	Score	Notes
A — MCQ	8		
B — Output Predict	8		
C — Code Trace	5		
D — Fill Blanks	4		
E — Error Fix	4		
F — Write Programs	10		
TOTAL	39		

Part A — Multiple Choice [2 marks each]

Q1. What is the type of args inside `def f(*args)` when called as `f(1, 2, 3)`?

(A) list (B) tuple (C) dict (D) set

Your answer: ()

Q2. Which function call correctly passes a dict as `**kwargs`?

(A) `f({"a":1})` (B) `f(**{"a":1})` (C) `f(*{"a":1})` (D) `f(dict(a=1))`

Your answer: ()

Q3. What does `func.__doc__` return?

(A) Source code of function (B) Function name (C) Docstring (if any) (D) Return type

Your answer: ()

Q4. What is the correct order of parameters in a function definition?

(A) `*args`, positional, `**kwargs` (B) positional, `**kwargs`, `*args` (C) positional, `*args`, `**kwargs` (D) `**kwargs`, `*args`, positional

Your answer: ()

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Expected
<pre>def f(a, b=10, *args, **kw): print(a, b, args, kw) f(1, 2, 3, 4, x=5)</pre>		

Code Snippet	Your Answer	Expected
<pre>def show(**kwargs): for k,v in sorted(kwargs.items()): print(f"{k}={v}") show(z=3, a=1, m=2)</pre>		
<pre>def f(*a, **k): return sum(a) + sum(k.values()) print(f(1,2,3, x=10, y=20))</pre>		
<pre>def g(x, /, y, *, z): return x+y+z print(g(1, y=2, z=3))</pre>		

Part C — Code Trace

Fill in each variable at each step of execution.

Q1. Trace the execution — fill in each variable at each step.

```
def mystery(*args):
    result = 1
    for n in args:
        result *= n
    return result

print(mystery(2, 3, 4))
```

Step	result	n	Output

Part D — Fill in the Blanks [1 mark each]

1. `**kwargs` inside a function is of type _____.
2. To access the docstring of a function `f`, use `f._____`.
3. Default arguments must appear _____ non-default arguments in the function definition.
4. To unpack a list `L` as positional arguments when calling `f`, use `f(_____L)`.

Part E — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix the function — it crashes when called.

```
def info(name, **details, age=0):  
    print(name, age, details)  
  
info("Raj", age=20, city="Pune")
```

Bug(s):

Corrected code:

Part F — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a function `describe(**kwargs)` that accepts any keyword arguments and prints them in the format "key: value", sorted alphabetically by key. Include a proper docstring.

Hint: Use `sorted(kwargs.items())` to iterate in alphabetical order.

Q2. Write a function `flexible_stats(*numbers)` that returns a dictionary with keys: `count`, `total`, `mean`, `minimum`, `maximum`. It should work for any number of arguments.

Hint: Use `len()`, `sum()`, `min()`, `max()` on the args tuple.

Topic 3 — String Methods (title, swapcase, maketrans...)

Self-Test · 42 marks

Section	Marks	Score	Notes
A — MCQ	8		
B — Output Predict	12		
C — Fill Blanks	4		
D — Error Fix	8		
E — Write Programs	10		
TOTAL	42		

Part A — Multiple Choice [2 marks each]

Q1. What does "the quick BROWN fox".title() return?

(A) The quick brown fox (B) THE QUICK BROWN FOX (C) The Quick Brown Fox (D) the quick brown fox

Your answer: ()

Q2. What is the result of "Hello World".capitalize()?

(A) Hello World (B) hello world (C) Hello world (D) HELLO WORLD

Your answer: ()

Q3. What does str.maketrans("abc", "xyz") do?

(A) Replaces a->x, b->y, c->z in any string that calls .translate() (B) Sorts the string (C) Reverses a,b,c to x,y,z (D) Returns a new string

Your answer: ()

Q4. "report.pdf".endswith((".pdf", ".docx", ".xlsx")) returns:

(A) Error — endswith takes one string (B) True (C) False (D) None

Your answer: ()

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Expected
<pre>s = "hElLo wOrld" print(s.swapcase())</pre>		
<pre>t = str.maketrans("aeiou", "12345") print("hello world".translate(t))</pre>		
<pre>print("hello".startswith(("he", "hi", "ho")))</pre>		

Code Snippet	Your Answer	Expected
<pre>print("Name: %s, Score: %05.1f" % ("Raj", 85.5))</pre>		
<pre>import re text = "cat bat rat" for m in re.finditer(r"[cbr]at", text): print(m.group(), m.start())</pre>		
<pre>print(" Python ".strip().lower().title())</pre>		

Part C — Fill in the Blanks [1 mark each]

1. `s.title()` capitalises the first letter of _____, while `s.capitalize()` capitalises only _____.
2. The third argument to `str.maketrans(f, t, d)` specifies characters to _____.
3. `re.finditer()` returns match objects with `.group()`, `.start()` and `._____` methods.
4. In % formatting, `%.2f` means a float with _____ decimal places.

Part D — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix the string formatting — it crashes.

```
name = "Raj"
age = 20
print("Name: %s, Age: %s, Score: %d" % (name, age))
```

Bug(s):

Corrected code:

2. The maketrans call should replace vowels with * but it does not work. Fix it.

```
s = "hello world"
table = "aeiou".maketrans("")
print(s.translate(table))
```

Bug(s):

Corrected code:

Part E — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a function `clean_text(s)` that: removes leading/trailing spaces, converts to title case, replaces all digits with #, and returns the result.

Hint: Use `strip()`, `title()`, `str.maketrans()` with digits, then `translate()`.

Q2. Write a program that uses `re.finditer()` to find all words in a string that start with a capital letter, and prints each word along with its start position.

Hint: Pattern: `r"[A-Z][a-z]*"` matches a capital followed by lowercase letters.

Topic 4 — Tuples — Packing, Unpacking & Operations

Self-Test · 29 marks

Section	Marks	Score	Notes
A — MCQ	8		
B — Output Predict	8		
C — Fill Blanks	4		
D — Error Fix	4		
E — Write Programs	5		
TOTAL	29		

Part A — Multiple Choice [2 marks each]

Q1. What is the type of `x = (5, )`?

(A) int (B) tuple (C) list (D) set

Your answer: ()

Q2. What does `a, *b, c = (1, 2, 3, 4, 5)` give for `b`?

(A) (2,3,4) (B) [2,3,4] (C) 2,3,4 (D) Error

Your answer: ()

Q3. What is the result of `(1, 2) > (1, 1, 9)`?

(A) True (B) False (C) Error (D) None

Your answer: ()

Q4. Which creates a genuine copy of tuple `t` as a new object?

(A) `t2 = t` (B) `t2 = t[:]` (C) `t2 = list(t)` (D) `t2 = t.copy()`

Your answer: ()

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Expected
<pre>a, b = 10, 20 a, b = b, a print(a, b)</pre>		
<pre>first, *rest = (1, 2, 3, 4, 5) print(type(rest), rest)</pre>		

Code Snippet	Your Answer	Expected
<pre>students = [("Raj", 85), ("Priya", 92), ("Sam", 78)] students.sort(key=lambda x: x[1]) print(students[0][0])</pre>		
<pre>x = (1, 2, 3) y = x x = x + (4,) print(y)</pre>		

Part C — Fill in the Blanks [1 mark each]

1. `a, *b = (1, 2, 3, 4)` gives `a = _____` and `b = _____`.
2. Tuples compare _____. The first unequal element determines the result.
3. To create an empty tuple, write _____.
4. Unlike lists, tuples have no `.copy()` method. Use _____ to create a copy.

Part D — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. This code should swap the first and last elements of a list using tuple unpacking. Fix it.

```
def swap_ends(lst):
    first, *mid, last = lst
    return last, *mid, first

print(swap_ends([1, 2, 3, 4, 5]))
```

Bug(s):

Corrected code:

Part E — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a function `flatten_pairs(pairs)` that takes a list of 2-tuples and returns two separate lists — one of all first elements and one of all second elements — using unpacking in a loop.

Hint: Use: `for first, second in pairs: ...`

Topic 5 — Sets — frozenset, Comprehension, all(), any()

Self-Test · 29 marks

Section	Marks	Score	Notes
A — MCQ	8		
B — Output Predict	8		
C — Fill Blanks	4		
D — Error Fix	4		
E — Write Programs	5		
TOTAL	29		

Part A — Multiple Choice [2 marks each]

Q1. Which is TRUE about frozenset?

(A) It is mutable (B) It can be used as a dict key (C) It supports .add() (D) It is ordered

Your answer: ()

Q2. all([]) returns:

(A) False (B) True (C) None (D) Error

Your answer: ()

Q3. What does {x*2 for x in range(4) if x%2==0} produce?

(A) {0, 4} (B) {0, 2, 4, 6} (C) [0, 4] (D) {0, 2}

Your answer: ()

Q4. any(x > 100 for x in [10, 20, 200, 5]) returns:

(A) False (B) [200] (C) True (D) 200

Your answer: ()

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Expected
<pre>fs1 = frozenset([1,2,3]) fs2 = frozenset([2,3,4]) print(fs1 & fs2)</pre>		
<pre>marks = [72, 85, 91, 60, 78] print(all(m >= 60 for m in marks)) print(any(m >= 90 for m in marks))</pre>		

Code Snippet	Your Answer	Expected
<pre>S = {x%3 for x in range(10)} print(sorted(S))</pre>		
<pre>d = {frozenset([1,2]): "A", frozenset([3]): "B"} print(d[frozenset([1,2])])</pre>		

Part C — Fill in the Blanks [1 mark each]

1. `any([])` returns _____ and `all([])` returns _____.
2. Set comprehension uses _____ braces, just like a set literal.
3. A frozenset supports set operations like `|` and `&` but raises `AttributeError` for _____ and _____.
4. `all()` short-circuits and stops at the first _____ value.

Part D — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix: trying to use a regular set as a dictionary key.

```
permissions = {}
role = {1, 2, 3}
permissions[role] = "admin"
print(permissions)
```

Bug(s):

Corrected code:

Part E — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a function `grade_summary(marks)` that takes a list of marks and uses `all()` and `any()` to print: whether all students passed (≥ 50), whether anyone got distinction (≥ 75), and a set of unique grade letters (A/B/C/F).

Hint: Use set comprehension for unique grades. `all()` and `any()` with generator expressions.

Topic 6 — Dictionary Comprehension & Advanced Dicts

Self-Test · 34 marks

Section	Marks	Score	Notes
A — MCQ	8		
B — Output Predict	8		
C — Fill Blanks	4		
D — Error Fix	4		
E — Write Programs	10		
TOTAL	34		

Part A — Multiple Choice [2 marks each]

Q1. What does `{v:k for k,v in {"a":1,"b":2}.items() }` produce?

(A) `{"a":1,"b":2}` (B) `{1:"a",2:"b"}` (C) `{1:2}` (D) Error

Your answer: ()

Q2. Which method merges d2 INTO d1 in-place?

(A) `d1 + d2` (B) `d1.update(d2)` (C) `d1 | d2` (D) `{**d1, **d2}`

Your answer: ()

Q3. In `{**d1, **d2}` when both have the same key, which wins?

(A) d1 (B) d2 (C) Error (D) Both are kept

Your answer: ()

Q4. What does `dict.items()` return?

(A) A list of keys (B) A list of values (C) A view of (key, value) tuples (D) A dict

Your answer: ()

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Expected
<pre>d = {"a":1,"b":2,"c":3,"d":4} filtered = {k:v for k,v in d.items() if v%2==0} print(filtered)</pre>		

Code Snippet	Your Answer	Expected
<pre>d1 = {"x":1, "y":2} d2 = {"y":20, "z":3} print(**d1, **d2)</pre>		
<pre>words = ["apple", "banana", "cherry", "avocado"] result = {w[0]: w for w in words} print(result)</pre>		
<pre>d = {"a":1, "b":2, "c":3} print(sorted(d, key=d.get, reverse=True))</pre>		

Part C — Fill in the Blanks [1 mark each]

1. Dict comprehension syntax: { ____: ____ for item in iterable if condition }.
2. In Python 3.9+, the ____ operator merges two dicts into a new one.
3. To iterate over both keys and values simultaneously, use for k, v in d. ____.
4. When two dicts are merged with {**d1, **d2}, the ____ dict wins on duplicate keys.

Part D — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix this word frequency counter — it crashes on the first word.

```
text = "the cat sat on the mat"
freq = {}
for word in text.split():
    freq[word] += 1
print(freq)
```

Bug(s):

Corrected code:

Part E — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a function `group_by_length(words)` that takes a list of words and returns a dictionary where keys are word lengths and values are lists of words with that length.

Hint: Use `setdefault()` or `dict.get()` to build lists. Or use a dict comprehension in two steps.

Q2. Using a single dict comprehension, write code that takes two lists `keys` and `values` and creates a dictionary from them, keeping only pairs where the value is greater than 0.

Hint: Use `zip(keys, values)` inside the comprehension.

Topic 7 — Modules — dir(), Namespaces, LEGB

Self-Test · 29 marks

Section	Marks	Score	Notes
A — MCQ	8		
B — Output Predict	8		
C — Fill Blanks	4		
D — Error Fix	4		
E — Write Programs	5		
TOTAL	29		

Part A — Multiple Choice [2 marks each]

Q1. When is the block `if __name__ == "__main__":` executed?

(A) Always (B) Only when imported (C) Only when run directly (D) Never — it is just a comment

Your answer: ()

Q2. What does `dir()` return when called with no arguments?

(A) All built-in functions (B) All installed modules (C) Names in the current local scope (D) Nothing

Your answer: ()

Q3. Python searches for a name in which order?

(A) Global -> Local -> Enclosing -> Built-in (B) Local -> Enclosing -> Global -> Built-in (C) Built-in -> Global -> Local (D) Local -> Global -> Built-in

Your answer: ()

Q4. After `from math import sqrt` AND then `from cmath import sqrt`:

(A) Both versions coexist (B) `cmath.sqrt` overwrites `math.sqrt` in local namespace (C) Error — name conflict (D) `math.sqrt` takes priority

Your answer: ()

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Expected
<pre># mymod.py content: # x = 42 # def greet(): return "hi" import mymod print(type(dir(mymod))) print("x" in dir(mymod))</pre>		
<pre>x = "global" def outer(): x = "outer" def inner(): print(x) inner() outer() print(x)</pre>		
<pre>import math as m print(m.pi > 3) print(hasattr(m, "sqrt"))</pre>		
<pre>x = 10 def f(): x = 20 return x print(f()) print(x)</pre>		

Part C — Fill in the Blanks [1 mark each]

1. Every Python file is a _____. Import it by its filename without the .py extension.
2. The LEGB rule stands for: Local, _____, Global, Built-in.
3. `import math as m` creates an _____ for the math module.
4. The `if __name__ == "__main__":` guard prevents code from running when the file is _____.

Part D — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix: the function should modify the global list, but it creates a local one instead.

```
results = []

def add_result(x):
    results = results + [x] # bug

add_result(10)
print(results)
```

Bug(s):

Corrected code:

Part E — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Create a module `stats.py` with functions `mean(data)`, `variance(data)`, and `std_dev(data)`. Include a `__name__ == "__main__"` block that tests each function. Show both the module file and how to import and use it.

Hint: `variance = mean of squared differences from the mean`. `std_dev = sqrt(variance)`.

Topic 8 — Scope, Lifetime, global, nonlocal, Closures

Self-Test · 35 marks

Section	Marks	Score	Notes
A — MCQ	8		
B — Output Predict	8		
C — Code Trace	6		
D — Fill Blanks	4		
E — Error Fix	4		
F — Write Programs	5		
TOTAL	35		

Part A — Multiple Choice [2 marks each]

Q1. What does `non local` allow an inner function to do?

- (A) Access global variables (B) Modify a variable in the enclosing (but not global) scope (C) Create a new local variable (D) Delete a variable from outer scope

Your answer: ()

Q2. When does a local variable get destroyed?

- (A) When the program ends (B) When it goes out of scope (function returns) (C) After 10 calls (D) Never — Python manages it

Your answer: ()

Q3. A closure keeps a captured variable alive because:

- (A) Python never deletes variables (B) The inner function holds a reference to the enclosing scope (C) `global` keyword is used (D) closures use lists internally

Your answer: ()

Q4. What is the output of: `def f(): x=1; x+=1; return x` called twice?

- (A) 2, 3 (B) 2, 2 (C) 1, 2 (D) Error

Your answer: ()

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write `ERROR` if it raises an exception.

Code Snippet	Your Answer	Expected
<pre>def make_adder(n): def add(x): return x + n return add add5 = make_adder(5) add10 = make_adder(10) print(add5(3), add10(3))</pre>		
<pre>count = 0 def inc(): global count count += 1 inc(); inc(); inc() print(count)</pre>		
<pre>def outer(): n = 0 def inner(): nonlocal n n += 1 return n return inner f = outer() print(f(), f(), f())</pre>		
<pre>x = "A" def f(): x = "B" def g(): return x return g() print(f(), x)</pre>		

Part C — Code Trace

Fill in each variable at each step of execution.

Q1. Trace the execution — fill in each variable at each step.

```
def counter_factory(start):
    count = start
    def increment(step=1):
        nonlocal count
        count += step
        return count
    return increment

c = counter_factory(10)
print(c())
print(c(5))
print(c())
```

Step	start	count	step	return	Output

Part D — Fill in the Blanks [1 mark each]

1. The LEGB lookup order is: Local, Enclosing, _____, Built-in.
2. A function that captures variables from its enclosing scope is called a _____.
3. To modify a global variable inside a function, use the _____ keyword before the variable name.
4. To modify an enclosing-scope variable from an inner function, use _____.

Part E — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix this attempt to create a counter using closures — it always returns 1.

```
def make_counter():  
    count = 0  
    def increment():  
        count = count + 1 # bug  
    return count  
    return increment  
  
c = make_counter()  
print(c())  
print(c())
```

Bug(s):

Corrected code:

Part F — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a closure-based `make_multiplier(n)` and use it to create `double`, `triple`, and `halve` functions. Then write a function `compose(f, g)` that returns a new function applying `g` then `f`.

Hint: `compose(f,g)(x) = f(g(x))`. A closure that captures both `f` and `g`.

— ANSWER KEY — Complete all 8 topics before opening this —

Topic 1 — Lambda, filter(), map(), reduce()

Answer Key · 34 marks

Part A — Multiple Choice [2 marks each]

Q1. What does `list(filter(None, [0, 1, "", "hi", False]))` return?

(A) `[0, 1, "", "hi", False]` (B) `[1, 'hi']` (C) `[None]` (D) Error

Answer: (B) `[1, 'hi']`

`filter(None, iterable)` keeps only truthy values. 0, "", and False are all falsy.

Q2. What is the output of `list(map(lambda x,y: x+y, [1,2], [10,20]))`?

(A) `[11, 22]` (B) `[1, 2, 10, 20]` (C) `[[1,10],[2,20]]` (D) Error

Answer: (A) `[11, 22]`

`map()` with two iterables pairs elements: `(1+10)=11`, `(2+20)=22`.

Q3. Which import is required to use `reduce()`?

(A) `from itertools import reduce` (B) `from functools import reduce` (C) `import reduce` (D) No import needed

Answer: (B) `from functools import reduce`

`reduce()` was moved to `functools` in Python 3. Must import: `from functools import reduce`.

Q4. `reduce(lambda a,b: a-b, [10,2,1])` evaluates as:

(A) `(10-2)-1 = 7` (B) `10-(2-1) = 9` (C) `10-2-1 = 7` (D) `-(10+2+1) = -13`

Answer: (A) `(10-2)-1 = 7`

`reduce` works left-to-right: first `lambda(10,2)=8`, then `lambda(8,1)=7`.

Part B — Fill in the Blanks [1 mark each]

- `filter()` returns a **filter object (iterator)** — you must wrap it with `list()` to see the values.
- `map()` applies a function to **every / each** element of the iterable.
- `reduce()` reduces an entire list to **a single** value.
- A lambda can have multiple arguments but only **one** expression.

Part C — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix the code to correctly compute the sum of squares of even numbers.

```
from functools import reduce
nums = [1,2,3,4,5,6]
result = reduce(lambda a,b: a+b,
map(lambda x: x**2,
filter(lambda x: x%2, nums)))
print(result)
```

Bug(s):

filter condition is wrong. `x%2` is truthy for ODD numbers (remainder 1). Should be `x%2==0` to keep evens.

Corrected code:

```
from functools import reduce
nums = [1,2,3,4,5,6]
result = reduce(lambda a,b: a+b,
map(lambda x: x**2,
filter(lambda x: x%2==0, nums)))
print(result) # 56 = 4+16+36
```

Part D — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Using only `lambda`, `map()`, `filter()`, and `reduce()`, write a one-expression pipeline that: takes a list of numbers, keeps only those divisible by 3, squares them, then returns their product.

Hint: Chain: `reduce(lambda..., map(lambda..., filter(lambda..., nums)))`

```
from functools import reduce
nums = [1, 2, 3, 4, 5, 6, 9, 12]
result = reduce(
lambda a, b: a * b,
map(lambda x: x**2,
filter(lambda x: x % 3 == 0, nums))
)
print(result) # 3^2 * 6^2 * 9^2 * 12^2 = 9*36*81*144 = 3779136
```

Q2. Write a function `apply_all(funcs, value)` that takes a list of functions and a value, and applies them in sequence (output of one is input of the next) using `reduce()`.

Hint: `reduce(lambda result, fn: fn(result), funcs, value)`

```
from functools import reduce

def apply_all(funcs, value):
    return reduce(lambda result, fn: fn(result), funcs, value)

# Test
operations = [
    lambda x: x * 2,
    lambda x: x + 10,
    lambda x: x ** 2,
]

print(apply_all(operations, 5)) # ((5*2)+10)^2 = 20^2 = 400
```

Topic 2 — Advanced Function Arguments & Docstrings

Answer Key · 39 marks

Part A — Multiple Choice [2 marks each]

Q1. What is the type of args inside `def f(*args)` when called as `f(1, 2, 3)`?

(A) list (B) tuple (C) dict (D) set

Answer: (B) tuple

*args collects extra positional arguments into a TUPLE, not a list.

Q2. Which function call correctly passes a dict as **kwargs?

(A) `f({"a":1})` (B) `f(**{"a":1})` (C) `f(*{"a":1})` (D) `f(dict(a=1))`

Answer: (B) `f(**{"a":1})`

**dict_name unpacks a dictionary into keyword arguments. `f(**{"a":1})` is equivalent to `f(a=1)`.

Q3. What does `func.__doc__` return?

(A) Source code of function (B) Function name (C) Docstring (if any) (D) Return type

Answer: (C) Docstring (if any)

`__doc__` is the docstring attribute — the first string literal inside a function/class body.

Q4. What is the correct order of parameters in a function definition?

(A) *args, positional, **kwargs (B) positional, **kwargs, *args (C) positional, *args, **kwargs (D) **kwargs, *args, positional

Answer: (C) positional, *args, **kwargs

Required order: positional args, then *args, then keyword-only args, then **kwargs.

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Correct Output
<pre>def f(a, b=10, *args, **kw): print(a, b, args, kw) f(1, 2, 3, 4, x=5)</pre>		1 2 (3, 4) {'x': 5}
<pre>def show(**kwargs): for k,v in sorted(kwargs.items()): print(f"{k}={v}") show(z=3, a=1, m=2)</pre>		a=1 m=2 z=3
<pre>def f(*a, **k): return sum(a) + sum(k.values()) print(f(1,2,3, x=10, y=20))</pre>		36
<pre>def g(x, /, y, *, z): return x+y+z print(g(1, y=2, z=3))</pre>		6

Part C — Code Trace

Fill in each variable at each step of execution.

Q1. Trace the execution — fill in each variable at each step.

```
def mystery(*args):
    result = 1
    for n in args:
        result *= n
    return result

print(mystery(2, 3, 4))
```

Step	result	n	Output
start	1	-	
n=2	2	2	
n=3	6	3	
n=4	24	4	
return	24	-	print: 24

Part D — Fill in the Blanks [1 mark each]

1. **kwargs inside a function is of type **dict**.
2. To access the docstring of a function f, use f. **__doc__**.
3. Default arguments must appear **after** non-default arguments in the function definition.
4. To unpack a list L as positional arguments when calling f, use f(***L**).

Part E — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix the function — it crashes when called.

```
def info(name, **details, age=0):
    print(name, age, details)

info("Raj", age=20, city="Pune")
```

Bug(s):

Keyword-only argument "age=0" cannot come after **kwargs. Keyword-only args must be placed BEFORE **kwargs.

Corrected code:

```
def info(name, age=0, **details):
    print(name, age, details)

info("Raj", age=20, city="Pune")
# Raj 20 {"city": "Pune"}
```

Part F — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a function `describe(**kwargs)` that accepts any keyword arguments and prints them in the format "key: value", sorted alphabetically by key. Include a proper docstring.

Hint: Use `sorted(kwargs.items())` to iterate in alphabetical order.

```
def describe(**kwargs):
    """
    Print keyword arguments in alphabetical order.

    Args:
        **kwargs: Any number of keyword arguments.

    Returns:
        None
    """
    for key in sorted(kwargs.keys()):
        print(f"{key}: {kwargs[key]}")

describe(city="Pune", name="Raj", age=20)
# age: 20
# city: Pune
# name: Raj
```

Q2. Write a function `flexible_stats(*numbers)` that returns a dictionary with keys: `count`, `total`, `mean`, `minimum`, `maximum`. It should work for any number of arguments.

Hint: Use `len()`, `sum()`, `min()`, `max()` on the args tuple.

```
def flexible_stats(*numbers):
    """
    Compute basic statistics for any number of values.
    Returns dict with count, total, mean, minimum, maximum.
    """
    if not numbers:
        return {"count": 0, "total": 0, "mean": 0, "minimum": None, "maximum": None}
    return {
        "count": len(numbers),
        "total": sum(numbers),
        "mean": sum(numbers) / len(numbers),
        "minimum": min(numbers),
        "maximum": max(numbers),
    }

print(flexible_stats(4, 7, 2, 9, 1))
# {count:5, total:23, mean:4.6, minimum:1, maximum:9}
```

Topic 3 — String Methods (title, swapcase, maketrans...)

Answer Key · 42 marks

Part A — Multiple Choice [2 marks each]

Q1. What does `"the quick BROWN fox".title()` return?

(A) The quick brown fox (B) THE QUICK BROWN FOX (C) The Quick Brown Fox (D) the quick brown fox

Answer: (C) The Quick Brown Fox

`title()` capitalizes the FIRST letter of EVERY word and lowercases the rest.

Q2. What is the result of `"Hello World".capitalize()`?

(A) Hello World (B) hello world (C) Hello world (D) HELLO WORLD

Answer: (C) Hello world

`capitalize()` makes only the FIRST character of the entire string uppercase; all others lowercase.

Q3. What does `str.maketrans("abc", "xyz")` do?

(A) Replaces a->x, b->y, c->z in any string that calls `.translate()` (B) Sorts the string (C) Reverses a,b,c to x,y,z (D) Returns a new string

Answer: (A) Replaces a->x, b->y, c->z in any string that calls .translate()

`maketrans` creates a translation TABLE mapping a->x, b->y, c->z. Apply with `s.translate(table)`.

Q4. `"report.pdf".endswith((".pdf", ".docx", ".xlsx"))` returns:

(A) Error — `endswith` takes one string (B) True (C) False (D) None

Answer: (B) True

`endswith()` (and `startswith()`) accept a TUPLE of suffixes/prefixes. Returns True if any match.

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Correct Output
<pre>s = "hElLo wOrLd" print(s.swapcase())</pre>		HeLIO wOrLd
<pre>t = str.maketrans("aeiou", "12345") print("hello world".translate(t))</pre>		h2ll4 w4rld
<pre>print("hello".startswith(("he", "hi", "ho")))</pre>		True
<pre>print("Name: %s, Score: %05.1f" % ("Raj", 85.5))</pre>		Name: Raj, Score: 085.5
<pre>import re text = "cat bat rat" for m in re.finditer(r"[cbr]at", text): print(m.group(), m.start())</pre>		cat 0 bat 4 rat 8

Code Snippet	Your Answer	Correct Output
<code>print(" Python ".strip().lower().title())</code>		Python

Part C — Fill in the Blanks [1 mark each]

- `s.title()` capitalises the first letter of **every word / the first character of the whole string**, while `s.capitalize()` capitalises only **every word / the first character of the whole string**.
- The third argument to `str.maketrans(f, t, d)` specifies characters to **delete**.
- `re.finditer()` returns match objects with `.group()`, `.start()` and `.end()` methods.
- In % formatting, `%.2f` means a float with **2** decimal places.

Part D — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix the string formatting — it crashes.

```
name = "Raj"
age = 20
print("Name: %s, Age: %s, Score: %d" % (name, age))
```

Bug(s):

3 format specifiers (%s, %s, %d) but only 2 values in the tuple. Must provide exactly 3 values.

Corrected code:

```
name = "Raj"
age = 20
score = 85
print("Name: %s, Age: %d, Score: %d" % (name, age, score))
# Name: Raj, Age: 20, Score: 85
```

2. The maketrans call should replace vowels with * but it does not work. Fix it.

```
s = "hello world"
table = "aeiou".maketrans("")
print(s.translate(table))
```

Bug(s):

maketrans is a static method on str, not an instance method. Also wrong call signature.

Corrected code:

```
s = "hello world"
table = str.maketrans("aeiou", "*****") # same-length strings
print(s.translate(table))

# h*ll* w*rld
```

Part E — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a function `clean_text(s)` that: removes leading/trailing spaces, converts to title case, replaces all digits with #, and returns the result.

Hint: Use `strip()`, `title()`, `str.maketrans()` with digits, then `translate()`.

```
import string

def clean_text(s):
    s = s.strip().title()
    table = str.maketrans(string.digits, "#" * len(string.digits))
    return s.translate(table)

print(clean_text(" hello world 2024 "))

# Hello World #####
```

Q2. Write a program that uses `re.finditer()` to find all words in a string that start with a capital letter, and prints each word along with its start position.

Hint: Pattern: `r"[A-Z][a-z]*"` matches a capital followed by lowercase letters.

```
import re

text = "The Quick brown Fox jumped Over the Fence"
for match in re.finditer(r"[A-Z][a-z]*", text):
    print(f"{match.group()!r} at position {match.start()}")
```

Topic 4 — Tuples — Packing, Unpacking & Operations

Answer Key · 29 marks

Part A — Multiple Choice [2 marks each]

Q1. What is the type of `x = (5, )`?

(A) int (B) tuple (C) list (D) set

Answer: (B) tuple

Trailing comma makes it a tuple. (5) without comma is just the integer 5.

Q2. What does `a, *b, c = (1, 2, 3, 4, 5)` give for `b`?

(A) (2,3,4) (B) [2,3,4] (C) 2,3,4 (D) Error

Answer: (B) [2,3,4]

Extended unpacking with `*` captures the middle elements as a LIST (not tuple).

Q3. What is the result of `(1, 2) > (1, 1, 9)`?

(A) True (B) False (C) Error (D) None

Answer: (A) True

Tuples compare lexicographically. At index 1: `2 > 1` is True — Python stops here and returns True.

Q4. Which creates a genuine copy of tuple `t` as a new object?

(A) `t2 = t` (B) `t2 = t[:]` (C) `t2 = list(t)` (D) `t2 = t.copy()`

Answer: (B) `t2 = t[:]`

`t2 = t` is an alias (same object). `t[:]` creates a new tuple with the same values. Note: tuples have no `.copy()` method.

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Correct Output
<pre>a, b = 10, 20 a, b = b, a print(a, b)</pre>		20 10
<pre>first, *rest = (1, 2, 3, 4, 5) print(type(rest), rest)</pre>		[2, 3, 4, 5]
<pre>students = [("Raj", 85), ("Priya", 92), ("Sam", 78)] students.sort(key=lambda x: x[1]) print(students[0][0])</pre>		Sam
<pre>x = (1, 2, 3) y = x x = x + (4,) print(y)</pre>		(1, 2, 3)

Part C — Fill in the Blanks [1 mark each]

1. `a, *b = (1, 2, 3, 4)` gives `a = 1` / `[2, 3, 4]` and `b = 1` / `[2, 3, 4]`.
2. Tuples compare **lexicographically (element by element)**. The first unequal element determines the result.
3. To create an empty tuple, write `()`.
4. Unlike lists, tuples have no `.copy()` method. Use `t[:]` or `tuple(t)` to create a copy.

Part D — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. This code should swap the first and last elements of a list using tuple unpacking. Fix it.

```
def swap_ends(lst):  
    first, *mid, last = lst  
    return last, *mid, first  
  
print(swap_ends([1, 2, 3, 4, 5]))
```

Bug(s):

Cannot use `*` in a return tuple literal. Must build the list and convert, or use a regular return.

Corrected code:

```
def swap_ends(lst):  
    first, *mid, last = lst  
    return [last] + mid + [first]  
  
print(swap_ends([1, 2, 3, 4, 5])) # [5, 2, 3, 4, 1]
```

Part E — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a function `flatten_pairs(pairs)` that takes a list of 2-tuples and returns two separate lists — one of all first elements and one of all second elements — using unpacking in a loop.

Hint: Use: `for first, second in pairs: ...`

```
def flatten_pairs(pairs):
    firsts, seconds = [], []
    for first, second in pairs:
        firsts.append(first)
        seconds.append(second)
    return firsts, seconds

names, scores = flatten_pairs([("Raj", 85), ("Priya", 92), ("Sam", 78)])
print(names) # ["Raj", "Priya", "Sam"]
print(scores) # [85, 92, 78]
```

Topic 5 — Sets — frozenset, Comprehension, all(), any()

Answer Key · 29 marks

Part A — Multiple Choice [2 marks each]

Q1. Which is TRUE about frozenset?

(A) It is mutable (B) It can be used as a dict key (C) It supports .add() (D) It is ordered

Answer: (B) It can be used as a dict key

frozenset is immutable and hashable, so it CAN be used as a dictionary key. Regular sets cannot.

Q2. all([]) returns:

(A) False (B) True (C) None (D) Error

Answer: (B) True

all([]) is True by definition — vacuous truth. There are no elements that are False.

Q3. What does {x*2 for x in range(4) if x%2==0} produce?

(A) {0, 4} (B) {0, 2, 4, 6} (C) [0, 4] (D) {0, 2}

Answer: (A) {0, 4}

range(4) = 0,1,2,3. Filter even: 0,2. Double: 0,4. Result is a set {0, 4}.

Q4. any(x > 100 for x in [10, 20, 200, 5]) returns:

(A) False (B) [200] (C) True (D) 200

Answer: (C) True

any() returns True as soon as one element satisfies the condition. 200 > 100 is True.

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Correct Output
<pre>fs1 = frozenset([1,2,3]) fs2 = frozenset([2,3,4]) print(fs1 & fs2)</pre>		frozenset({2, 3})
<pre>marks = [72, 85, 91, 60, 78] print(all(m >= 60 for m in marks)) print(any(m >= 90 for m in marks))</pre>		True True
<pre>S = {x%3 for x in range(10)} print(sorted(S))</pre>		[0, 1, 2]
<pre>d = {frozenset([1,2]): "A", frozenset([3]): "B"} print(d[frozenset([1,2])])</pre>		A

Part C — Fill in the Blanks [1 mark each]

1. `any([ ])` returns **False / True** and `all([ ])` returns **False / True**.
2. Set comprehension uses **curly { }** braces, just like a set literal.
3. A frozenset supports set operations like `|` and `&` but raises `AttributeError` for `.add()` / `.remove()` and `.add()` / `.remove()`.
4. `all()` short-circuits and stops at the first **False (falsy)** value.

Part D — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix: trying to use a regular set as a dictionary key.

```
permissions = {}
role = {1, 2, 3}
permissions[role] = "admin"
print(permissions)
```

Bug(s):

Regular sets are mutable and unhashable — they cannot be dict keys. `TypeError: unhashable type: 'set'`

Corrected code:

```
permissions = {}
role = frozenset({1, 2, 3}) # frozenset is hashable
permissions[role] = "admin"
print(permissions)
# {frozenset({1, 2, 3}): "admin"}
```

Part E — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a function `grade_summary(marks)` that takes a list of marks and uses `all()` and `any()` to print: whether all students passed (≥ 50), whether anyone got distinction (≥ 75), and a set of unique grade letters (A/B/C/F).

Hint: Use set comprehension for unique grades. `all()` and `any()` with generator expressions.

```
def grade_summary(marks):
    print("All passed:", all(m >= 50 for m in marks))
    print("Distinction exists:", any(m >= 75 for m in marks))
    grades = {"A" if m >= 75 else "B" if m >= 60 else "C" if m >= 50 else "F"}
    for m in marks:
        pass
    print("Unique grades:", grades)

grade_summary([82, 45, 67, 91, 55])

# All passed: False
# Distinction exists: True
# Unique grades: {'A', 'B', 'C', 'F'}
```

Topic 6 — Dictionary Comprehension & Advanced Dicts

Answer Key · 34 marks

Part A — Multiple Choice [2 marks each]

Q1. What does `{v:k for k,v in {"a":1,"b":2}.items()}` produce?

(A) `{"a":1,"b":2}` (B) `{1:"a",2:"b"}` (C) `{1:2}` (D) Error

Answer: (B) `{1:"a",2:"b"}`

Dict comprehension swapping keys and values: `{1:"a", 2:"b"}`.

Q2. Which method merges d2 INTO d1 in-place?

(A) `d1 + d2` (B) `d1.update(d2)` (C) `d1 | d2` (D) `{**d1, **d2}`

Answer: (B) `d1.update(d2)`

`update()` modifies d1 in-place. | and `{**d1,**d2}` create a NEW dictionary without changing d1.

Q3. In `{**d1, **d2}` when both have the same key, which wins?

(A) d1 (B) d2 (C) Error (D) Both are kept

Answer: (B) d2

d2 values overwrite d1 values for duplicate keys — later dict wins.

Q4. What does `dict.items()` return?

(A) A list of keys (B) A list of values (C) A view of (key, value) tuples (D) A dict

Answer: (C) A view of (key, value) tuples

`.items()` returns a `dict_items` view — iterable of (key, value) tuples. Not a list, but usable in loops.

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Correct Output
<pre>d = {"a":1,"b":2,"c":3,"d":4} filtered = {k:v for k,v in d.items() if v%2==0} print(filtered)</pre>		<code>{'b': 2, 'd': 4}</code>
<pre>d1 = {"x":1, "y":2} d2 = {"y":20, "z":3} print(**d1, **d2)</pre>		<code>{'x': 1, 'y': 20, 'z': 3}</code>
<pre>words = ["apple","banana","cherry","avocado"] result = {w[0]: w for w in words} print(result)</pre>		<code>{'a': 'avocado', 'b': 'banana', 'c': 'cherry'}</code>
<pre>d = {"a":1,"b":2,"c":3} print(sorted(d, key=d.get, reverse=True))</pre>		<code>['c', 'b', 'a']</code>

Part C — Fill in the Blanks [1 mark each]

1. Dict comprehension syntax: { **key_expression** : **value_expression**: **key_expression** : **value_expression** for item in iterable if condition }.
2. In Python 3.9+, the | **(pipe)** operator merges two dicts into a new one.
3. To iterate over both keys and values simultaneously, use for k, v in d.**items()**.
4. When two dicts are merged with {**d1, **d2}, the **second (d2)** dict wins on duplicate keys.

Part D — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix this word frequency counter — it crashes on the first word.

```
text = "the cat sat on the mat"
freq = {}
for word in text.split():
    freq[word] += 1
print(freq)
```

Bug(s):

freq[word] += 1 raises KeyError when word is not yet in freq. Must initialise with get().

Corrected code:

```
text = "the cat sat on the mat"
freq = {}
for word in text.split():
    freq[word] = freq.get(word, 0) + 1
print(freq)
```

Part E — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a function `group_by_length(words)` that takes a list of words and returns a dictionary where keys are word lengths and values are lists of words with that length.

Hint: Use `setdefault()` or `dict.get()` to build lists. Or use a dict comprehension in two steps.

```
def group_by_length(words):
    result = {}
    for word in words:
        length = len(word)
        result.setdefault(length, []).append(word)
    return result

words = ["cat", "dog", "elephant", "ant", "bear", "eagle"]
print(group_by_length(words))

# {3:['cat', 'dog', 'ant'], 8:['elephant'], 4:['bear'], 5:['eagle']}
```

Q2. Using a single dict comprehension, write code that takes two lists `keys` and `values` and creates a dictionary from them, keeping only pairs where the value is greater than 0.

Hint: Use `zip(keys, values)` inside the comprehension.

```
keys = ["a", "b", "c", "d", "e"]
values = [10, -5, 0, 7, -2]

result = {k: v for k, v in zip(keys, values) if v > 0}
print(result) # {"a": 10, "d": 7}
```

Topic 7 — Modules — dir(), Namespaces, LEGB

Answer Key · 29 marks

Part A — Multiple Choice [2 marks each]

Q1. When is the block `if __name__ == "__main__":` executed?

(A) Always (B) Only when imported (C) Only when run directly (D) Never — it is just a comment

Answer: (C) Only when run directly

`__name__` equals `"__main__"` only when the file is run directly. When imported, `__name__` is the module name.

Q2. What does `dir()` return when called with no arguments?

(A) All built-in functions (B) All installed modules (C) Names in the current local scope (D) Nothing

Answer: (C) Names in the current local scope

`dir()` with no argument returns a list of names in the current local scope.

Q3. Python searches for a name in which order?

(A) Global -> Local -> Enclosing -> Built-in (B) Local -> Enclosing -> Global -> Built-in (C) Built-in -> Global -> Local (D) Local -> Global -> Built-in

Answer: (B) Local -> Enclosing -> Global -> Built-in

LEGB rule: Local -> Enclosing -> Global -> Built-in. Python checks innermost scope first.

Q4. After `from math import sqrt` AND then `from cmath import sqrt`:

(A) Both versions coexist (B) `cmath.sqrt` overwrites `math.sqrt` in local namespace (C) Error — name conflict (D) `math.sqrt` takes priority

Answer: (B) `cmath.sqrt` overwrites `math.sqrt` in local namespace

`from X import sqrt` copies the name `sqrt` into the current namespace. The second import REPLACES it.

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write ERROR if it raises an exception.

Code Snippet	Your Answer	Correct Output
<pre># mymod.py content: # x = 42 # def greet(): return "hi" import mymod print(type(dir(mymod))) print("x" in dir(mymod))</pre>		True
<pre>x = "global" def outer(): x = "outer" def inner(): print(x) inner() outer() print(x)</pre>		outer global

Code Snippet	Your Answer	Correct Output
<pre>import math as m print(m.pi > 3) print(hasattr(m, "sqrt"))</pre>		True True
<pre>x = 10 def f(): x = 20 return x print(f()) print(x)</pre>		20 10

Part C — Fill in the Blanks [1 mark each]

1. Every Python file is a **module**. Import it by its filename without the `.py` extension.
2. The LEGB rule stands for: Local, **Enclosing**, Global, Built-in.
3. `import math as m` creates an **alias** for the `math` module.
4. The `if __name__ == "__main__":` guard prevents code from running when the file is **imported (as a module)**.

Part D — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. **Fix:** the function should modify the global list, but it creates a local one instead.

```
results = []

def add_result(x):
    results = results + [x] # bug
    add_result(10)
    print(results)
```

Bug(s):

results = results + [x] treats "results" as local (due to assignment) but reads it before assigning — UnboundLocalError. Use global or results.append().

Corrected code:

```
results = []

def add_result(x):
    results.append(x) # modifies in-place — no need for global
    add_result(10)
    add_result(20)
    print(results) # [10, 20]
```

Part E — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Create a module `stats.py` with functions `mean(data)`, `variance(data)`, and `std_dev(data)`. Include a `__name__ == "__main__"` block that tests each function. Show both the module file and how to import and use it.

Hint: `variance` = mean of squared differences from the mean. `std_dev` = `sqrt(variance)`.

```
# stats.py
import math

def mean(data):
    """Return arithmetic mean of data."""
    return sum(data) / len(data)

def variance(data):
    """Return population variance of data."""
    m = mean(data)
    return sum((x - m)**2 for x in data) / len(data)

def std_dev(data):
    """Return standard deviation of data."""
    return math.sqrt(variance(data))

if __name__ == "__main__":
    sample = [4, 7, 2, 9, 1, 6]
    print("Mean:", mean(sample))
    print("Variance:", round(variance(sample), 2))
    print("Std Dev:", round(std_dev(sample), 2))

# Usage from another file:
# import stats
# print(stats.mean([1,2,3,4,5]))
```

Topic 8 — Scope, Lifetime, global, nonlocal, Closures

Answer Key · 35 marks

Part A — Multiple Choice [2 marks each]

Q1. What does `nonlocal` allow an inner function to do?

- (A) Access global variables (B) Modify a variable in the enclosing (but not global) scope (C) Create a new local variable (D) Delete a variable from outer scope

Answer: (B) Modify a variable in the enclosing (but not global) scope

`nonlocal` tells Python the variable is in the nearest enclosing scope (not global). It allows modification.

Q2. When does a local variable get destroyed?

- (A) When the program ends (B) When it goes out of scope (function returns) (C) After 10 calls (D) Never — Python manages it

Answer: (B) When it goes out of scope (function returns)

Local variables are created when the function is called and destroyed when it returns (goes out of scope).

Q3. A closure keeps a captured variable alive because:

- (A) Python never deletes variables (B) The inner function holds a reference to the enclosing scope (C) `global` keyword is used (D) closures use lists internally

Answer: (B) The inner function holds a reference to the enclosing scope

The inner function object holds a reference to the cell object containing the variable, keeping it alive.

Q4. What is the output of: `def f(): x=1; x+=1; return x` called twice?

- (A) 2, 3 (B) 2, 2 (C) 1, 2 (D) Error

Answer: (B) 2, 2

Local variable `x` is created fresh every call. Each call: `x=1`, `x=2`, return 2. Both calls return 2.

Part B — Predict the Output [2 marks each]

Write exactly what the code prints. Write `ERROR` if it raises an exception.

Code Snippet	Your Answer	Correct Output
<pre>def make_adder(n): def add(x): return x + n return add add5 = make_adder(5) add10 = make_adder(10) print(add5(3), add10(3))</pre>		8 13
<pre>count = 0 def inc(): global count count += 1 inc(); inc(); inc() print(count)</pre>		3

Code Snippet	Your Answer	Correct Output
<pre>def outer(): n = 0 def inner(): nonlocal n n += 1 return n return inner f = outer() print(f(), f(), f())</pre>		1 2 3
<pre>x = "A" def f(): x = "B" def g(): return x return g() print(f(), x)</pre>		B A

Part C — Code Trace

Fill in each variable at each step of execution.

Q1. Trace the execution — fill in each variable at each step.

```
def counter_factory(start):
    count = start
    def increment(step=1):
        nonlocal count
        count += step
        return count
    return increment

c = counter_factory(10)
print(c())
print(c(5))
print(c())
```

Step	start	count	step	return	Output
factory(10)	10	10	-	-	
c() — step=1	-	11	1	11	
c(5) — step=5	-	16	5	16	
c() — step=1	-	17	1	17	
output	-	-	-	11 then 16 then 17	

Part D — Fill in the Blanks [1 mark each]

1. The LEGB lookup order is: Local, Enclosing, **Global**, Built-in.
2. A function that captures variables from its enclosing scope is called a **closure**.
3. To modify a global variable inside a function, use the **global** keyword before the variable name.
4. To modify an enclosing-scope variable from an inner function, use **nonlocal**.

Part E — Find & Fix the Error [4 marks each]

State the bug clearly, then write the complete corrected code.

1. Fix this attempt to create a counter using closures — it always returns 1.

```
def make_counter():
    count = 0
    def increment():
        count = count + 1 # bug
    return count
    return increment

c = make_counter()
print(c())
print(c())
```

Bug(s):

count = count + 1 makes Python treat count as local (due to assignment). UnboundLocalError. Must use nonlocal.

Corrected code:

```
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count = count + 1
    return count
    return increment

c = make_counter()
print(c()) # 1
print(c()) # 2
```

Part F — Write Programs [5 marks each]

Write clean, working code. Comment your logic where helpful.

Q1. Write a closure-based `make_multiplier(n)` and use it to create `double`, `triple`, and `halve` functions. Then write a function `compose(f, g)` that returns a new function applying `g` then `f`.

Hint: `compose(f,g)(x) = f(g(x))`. A closure that captures both `f` and `g`.

```
def make_multiplier(n):
    def multiply(x):
        return x * n
    return multiply

double = make_multiplier(2)
triple = make_multiplier(3)
halve = make_multiplier(0.5)

print(double(5)) # 10
print(triple(4)) # 12
print(halve(10)) # 5.0

def compose(f, g):
    def composed(x):
        return f(g(x))
    return composed

double_then_triple = compose(triple, double)
print(double_then_triple(4)) # triple(double(4)) = triple(8) = 24
```